

Documento C — Técnicas e Casos de Teste

Projeto: Sidera Predict
Tecnologia: Flutter
Arquitetura: MVVM
Norma aplicada: ISO/IEC/IEEE 29119-4

1. Técnicas Utilizadas

Técnica	Finalidade
Particionamento de Equivalência	Separar entradas válidas e inválidas
Valor Limite	Validar campos vazios e estados limítrofes
Transição de Estado	Validar mudança de status (IA, conformidade)
Teste Baseado em Cenário	Validar fluxo completo entre telas/etapas
Teste de Desempenho Funcional	Validar resposta da UI mesmo com serviço remoto lento

2. Derivação das Condições de Teste

CT01 — Validar login com credenciais válidas

Técnica: Particionamento de Equivalência
Justificativa: Existe uma classe válida: e-mail correto + senha correta.

TC01 — Login com credenciais válidas

Entradas: email = "[joao@email.com](#)", senha = "Senha@123"
Resultado Esperado: retorno `true`, `userName` = "João"

CT02 — Validar login com senha incorreta

Técnica: Particionamento de Equivalência
Justificativa: Senha incorreta representa classe inválida.

TC02 — Login com senha errada

Entradas: email = "joao@email.com", senha = "senhaerrada"

Resultado Esperado: retorno `false`, `errorMessage` contendo "Credenciais"

CT03 — Validar login com usuário inexistente

Técnica: Particionamento de Equivalência

Justificativa: E-mail não cadastrado = classe inválida.

TC03 — Login com usuário inexistente

Entradas: email = "naoexiste@email.com"

Resultado Esperado: retorno `false`, `errorMessage` presente

CT04 — Validar estado de loading

Técnica: Valor Limite

Justificativa: Verifica estado nos limites (antes/depois da operação).

TC04 — isLoading antes e depois do login

Resultado Esperado: `isLoading` = `false` em ambos os momentos

CT05 — Validar cadastro com dados válidos

Técnica: Particionamento de Equivalência

Justificativa: Classe válida: todos os campos preenchidos corretamente.

TC05 — Cadastro com dados válidos

Entradas: email, senha, matrícula, nome

Resultado Esperado: retorno `true`, `userName` preenchido

CT06 — Validar cadastro com e-mail duplicado

Técnica: Transição de Estado

Justificativa: Sistema muda de "e-mail disponível" para "e-mail já em uso".

TC06 — E-mail duplicado

Pré-condição: Usuário já cadastrado com o e-mail.

Resultado Esperado: retorno `false`, erro contendo "e-mail"

CT07 — Validar cadastro com matrícula duplicada

Técnica: Transição de Estado

Justificativa: Sistema muda de "matrícula disponível" para "já cadastrada".

TC07 — Matrícula duplicada

Pré-condição: Matrícula já cadastrada.

Resultado Esperado: retorno `false`, erro contendo "matrícula"

CT08 — Validar verificação de matrícula

Técnica: Particionamento de Equivalência

Justificativa: Matrícula livre (válida) vs. matrícula ocupada (inválida).

TC08 — Matrícula disponível/indisponível

Resultado Esperado: `true` para livre, `false` para ocupada

CT09 — Validar verificação de e-mail

Técnica: Particionamento de Equivalência

Justificativa: E-mail livre (válido) vs. e-mail ocupado (inválido).

TC09 — E-mail disponível/indisponível

Resultado Esperado: `true` para livre, `false` para ocupado

CT10 — Validar status "Na Fila" da IA

Técnica: Transição de Estado

Justificativa: Estado inicial do relatório = `pending`.

TC10 — Status pending

Resultado Esperado: `aiReportStatus == pending`, `isAiReportStreaming == true`

CT11 — Validar status "Gerando" da IA

Técnica: Transição de Estado

Justificativa: Durante processamento = `generating`.

TC11 — Status generating

Resultado Esperado: `aiReportStatus == generating`, `isAiReportStreaming == true`

CT12 — Validar status "Concluído" da IA

Técnica: Transição de Estado

Justificativa: Após conclusão = `completed`, relatório preenchido.

TC12 — Status completed

Resultado Esperado: `aiReportStatus == completed`, `isAiReportStreaming == false`, `aiReport` não vazio

CT13 — Validar mensagem de falha na calibração ArUco

Técnica: Particionamento de Equivalência

Justificativa: `calibrationSuccess == false` = classe de erro.

TC13 — Falha na calibração

Resultado Esperado: mensagem contendo "calibração" e "ArUco"

CT14 — Validar mensagem de peça não encontrada

Técnica: Particionamento de Equivalência

Justificativa: `objectFound == false` = classe de erro.

TC14 — Peça não encontrada

Resultado Esperado: mensagem contendo "não encontrada"

CT15 — Validar mensagem padrão (imagem nula)

Técnica: Valor Limite

Justificativa: Draft `null` = caso limite.

TC15 — Imagem nula

Resultado Esperado: mensagem contendo "processar"

CT16 — Validar mensagem personalizada

Técnica: Particionamento de Equivalência

Justificativa: Quando `lastError` é fornecido, deve ser usado diretamente.

TC16 — Imagem nula com lastError

Resultado Esperado: retorna exatamente o `lastError` informado

CT17 — Validar draft válido

Técnica: Particionamento de Equivalência

Justificativa: Draft com calibração + peça + dados = classe válida.

TC17 — Draft válido

Resultado Esperado: `isValidMeasurement == true`

CT18 — Validar draft sem calibração

Técnica: Particionamento de Equivalência

Justificativa: Sem calibração = classe inválida.

TC18 — Draft inválido

Resultado Esperado: `isValidMeasurement == false`

CT19 — Validar criação de registro completo

Técnica: Teste Baseado em Cenário

Justificativa: Cenário: criar registro com todos os campos preenchidos.

TC19 — Registro completo

Resultado Esperado: todos os campos do `MeasurementRecord` preenchidos

CT20 — Validar draft sem peça encontrada

Técnica: Particionamento de Equivalência

Justificativa: `objectFound == false` = inválido para salvar.

TC20 — Draft sem peça

Resultado Esperado: `isValidMeasurement == false`, `objectFound == false`

CT21 — Validar conformidade OK

Técnica: Particionamento de Equivalência

Justificativa: Status OK = sem campos de reprovação.

TC21 — Conformidade OK

Resultado Esperado: `nonConformityReason == null`

CT22 — Validar conformidade NOK

Técnica: Particionamento de Equivalência

Justificativa: Status NOK = com motivo de reprovação.

TC22 — Conformidade NOK

Resultado Esperado: `nonConformityReason` preenchido

CT23 — Validar copyWith

Técnica: Valor Limite

Justificativa: Apenas campos alterados devem mudar.

TC23 — copyWith mantém dados

Resultado Esperado: campo alterado muda, demais permanecem iguais

CT24 — Validar serialização JSON

Técnica: Teste Baseado em Cenário

Justificativa: Cenário: converter para JSON e restaurar sem perda de dados.

TC24 — JSON ida e volta

Resultado Esperado: todos os campos restaurados iguais ao original

3. Casos de Integração de UI

Os casos abaixo são executados com `integration_test`, simulando interação real com a interface Flutter em dispositivo/emulador.

IT-TC25 — Cadastro e navegação para Menu Principal

Técnica: Teste Baseado em Cenário

Justificativa: Valida o fluxo completo de cadastro, Provider, ViewModel fake e navegação.

Resultado Esperado: cadastro realizado, `userName` atualizado e tela "MENU PRINCIPAL" exibida.

IT-TC26 — Login com campos vazios

Técnica: Valor Limite

Justificativa: Campos vazios representam o limite inferior de entrada do formulário.

Resultado Esperado: mensagens "Campo obrigatório" e "Senha obrigatória" exibidas, sem navegação.

IT-TC27 — Login inválido

Técnica: Particionamento de Equivalência

Justificativa: Senha incorreta representa classe inválida de autenticação.

Resultado Esperado: mensagem "Credenciais inválidas. Verifique seus dados." exibida e Menu Principal não aberto.

IT-TC28 — Login válido

Técnica: Particionamento de Equivalência

Justificativa: Usuário cadastrado e senha correta representam classe válida de autenticação.

Resultado Esperado: login realizado, `userName` atualizado e tela "MENU PRINCIPAL" exibida.

IT-TC29 — Fluxo completo com modos, medição, anotação e IA

Técnica: Teste Baseado em Cenário + Transição de Estado

Justificativa: Valida o fluxo ponta a ponta da interface e a transição do relatório IA dentro do fluxo de medição.

Passos principais:

1. Realizar login válido.
2. Acessar configurações.
3. Ativar modo escuro.
4. Ativar alto contraste.
5. Retornar ao modo claro.
6. Acessar histórico vazio.
7. Abrir nova medição com câmera simulada.
8. Processar medição simulada válida.
9. Preencher identificação da peça.

10. Marcar "NÃO CONFORME".
11. Selecionar todos os motivos de reprovação.
12. Registrar observação/anotação.
13. Salvar medição e validar animação "MEDIÇÃO SALVA".
14. Validar IA em fila (`pending`).
15. Validar IA gerando (`generating`).
16. Liberar relatório final fake.
17. Validar IA concluída (`completed`).
18. Abrir histórico e confirmar registro cadastrado.

Resultado Esperado: registro salvo no histórico, status "NÃO CONFORME" exibido, anotação persistida e ciclo IA `pending` → `generating` → `completed` concluído.

CT30 — Validar salvamento responsivo com nuvem lenta

Técnica: Teste de Desempenho Funcional + Teste Baseado em Cenário

Justificativa: O app é multiusuário e deve priorizar a nuvem, mas a interface não pode ficar bloqueada enquanto Supabase/Storage respondem lentamente.

TC30 — Salvamento local antes da sincronização remota

Entradas: draft válido, serviço remoto fake com atraso controlado.

Resultado Esperado: `saveMeasurement` retorna rapidamente com registro local persistido, enquanto a tentativa remota ocorre em segundo plano.

CT31 — Validar otimização Full HD sem distorção

Técnica: Valor Limite + Particionamento de Equivalência

Justificativa: Imagens acima de Full HD devem ser reduzidas ao limite máximo sem alterar a proporção original.

TC31 — Imagem grande otimizada para JPEG proporcional

Entradas: imagem `4000x2000` .

Resultado Esperado: saída JPEG com dimensões dentro de `1920x1080` , proporção preservada e sem corte/esticamento.

4. Tabela Consolidada de Técnicas

Condição	Técnica
CT01	Particionamento
CT02	Particionamento
CT03	Particionamento
CT04	Valor Limite
CT05	Particionamento
CT06	Transição de Estado
CT07	Transição de Estado
CT08	Particionamento
CT09	Particionamento
CT10	Transição de Estado
CT11	Transição de Estado
CT12	Transição de Estado
CT13	Particionamento
CT14	Particionamento
CT15	Valor Limite
CT16	Particionamento
CT17	Particionamento
CT18	Particionamento
CT19	Cenário
CT20	Particionamento
CT21	Particionamento
CT22	Particionamento
CT23	Valor Limite
CT24	Cenário
CT30	Desempenho Funcional + Cenário
CT31	Valor Limite + Particionamento

5. Tabela Consolidada de Casos de Teste

ID	Caso de Teste
TC01	Login com credenciais válidas
TC02	Login com senha errada
TC03	Login com usuário inexistente
TC04	Estado de loading durante login
TC05	Cadastro com dados válidos
TC06	Cadastro com e-mail duplicado
TC07	Cadastro com matrícula duplicada
TC08	Verificação de matrícula disponível
TC09	Verificação de e-mail disponível
TC10	Status "Na Fila" da IA
TC11	Status "Gerando" da IA
TC12	Status "Concluído" da IA
TC13	Falha na calibração ArUco
TC14	Peça não encontrada
TC15	Imagem nula (erro genérico)
TC16	Imagem nula com lastError personalizado
TC17	Draft válido para salvar
TC18	Draft sem calibração (inválido)
TC19	Registro criado com dados completos
TC20	Draft sem peça encontrada (inválido)
TC21	Conformidade OK sem reprovação
TC22	Conformidade NOK com motivo
TC23	copyWith mantendo dados originais
TC24	Serialização JSON ida e volta
TC25	Cadastro → Login → Verificar usuário
TC26	Login inválido → Re-tentativa correta
TC27	Cadastro duplicado → Login original
TC28	Ciclo de status da IA completo

ID	Caso de Teste
TC29	Serializar e restaurar lista de registros
TC30	Salvamento local responsivo com nuvem lenta
TC31	Otimização JPEG Full HD sem distorção

6. Tabela Consolidada de Integração de UI

ID	Caso de Teste
IT-TC25	Cadastro e navegação para Menu Principal
IT-TC26	Login com campos vazios exibindo validações
IT-TC27	Login inválido exibindo mensagem e sem navegação
IT-TC28	Login válido navegando para Menu Principal
IT-TC29	Auth → modos visuais → histórico → medição simulada → anotação → animação de sucesso → IA pending/generating/completed → histórico

7. Conclusão da Etapa

As condições de teste identificadas no Documento A foram derivadas em 31 casos de teste automatizados em `test/` e 5 casos de integração de UI em `integration_test/`, utilizando técnicas formais definidas pela ISO/IEC/IEEE 29119-4.

Os casos de teste produzidos estão implementados como testes automatizados no projeto Flutter, utilizando testes de unidade e testes de integração.